SENTINELAI

Intelligent Cyber Threat Detection System

FINAL-YEAR B.TECH COMPUTER SCIENCE & ENGINEERING PROJECT

A defensive security analytics platform that analyzes security-event records, detects anomalous patterns, estimates threat categories, prioritizes alerts, and presents results through a web dashboard.

Layer	Technology
Frontend	React + Vite
Backend	Java 17 + Spring Boot
AI/ML	Python + FastAPI + Scikit-learn
Database	MySQL
Testing	JUnit + Pytest
Deployment	Docker Compose (planned)

Project report and GitHub documentation | Version 1.0 | 2026

Declaration and Project Scope

This report describes SentinelAI, a defensive cybersecurity analytics prototype for academic demonstration. It analyzes supplied security-event records and synthetic examples. It does not conduct attacks, exploit vulnerabilities, scan networks, or access systems.

Submission note: Add your name, roll number, department, college, supervisor, and academic year. Verify the implementation and test results before submission. This PDF is a project report/specification, not the complete runnable source-code repository; commit the source folders alongside it.

Abstract

SentinelAI combines a Java Spring Boot application, Python machine-learning service, MySQL database, and React dashboard. It helps an analyst review structured event records, identify unusual behavior, categorize potential threats, and prioritize alerts. The ML service demonstrates anomaly detection and optional supervised classification. The backend coordinates validation, persistence, and workflows; the dashboard presents metrics, event tables, filters, and alert details. Model output is advisory, not proof of malicious activity.

Keywords: Cybersecurity analytics, anomaly detection, Random Forest, Isolation Forest, Spring Boot, FastAPI, React, MySQL.

1. Introduction

Security systems generate large volumes of events from applications, servers, endpoints, and network monitoring tools. Manual review can be difficult. SentinelAI demonstrates a defensive pipeline to normalize event attributes, estimate anomalousness, prioritize review, and present the result in one interface.

1.1 Problem statement and objectives

Problem: provide an accessible academic prototype for defensive security-event triage using a modern full-stack architecture and ML methods.

• Build a React dashboard for event summaries and alert review.

• Develop Java REST APIs for validation, persistence, and alert workflows.

• Create a Python service for anomaly scoring and threat-category estimation.

• Store event metadata, model output, and analyst status in MySQL.

• Provide reproducible setup, test plans, documentation, and GitHub version control.

1.2 Scope

In scope: analysis of pre-collected authorized records, synthetic test data, anomaly scoring, alert prioritization, dashboards, and analyst status updates. **Out of scope:** exploitation, offensive operations, credential collection, covert monitoring, or automatic blocking.

2. System Architecture

React communicates with the Java backend. Spring Boot validates event records, stores them in MySQL, and calls the Python inference service. Python returns an anomaly score and optional category estimate. The backend stores the result and exposes it to the dashboard.

Layer	Technology	Responsibility
Presentation	React + Vite	Dashboard, event table, filters, alert details
Application	Java 17 + Spring Boot	REST API, validation, workflows, persistence
ML service	Python + FastAPI + Scikit-learn	Feature preparation, inference, scoring

Layer	Technology	Responsibility
Data	MySQL	Events, alerts, model metadata

2.1 Data flow

• User submits a permitted event record.

• Spring Boot validates the request and stores the event.

• Backend sends approved features to the ML service.

• Python applies the saved preprocessing and model.

• ML service returns a score, category estimate, and model version.

• Backend stores the analysis and derives a documented risk band.

• Analyst reviews the alert and updates its status.

2.2 Suggested repository structure

```
sentinelai/
  frontend/                React + Vite
  backend/                 Java 17 + Spring Boot
  ml-service/              Python + FastAPI + Scikit-learn
    app/
    tests/
    data/
  database/schema.sql
  docs/
    SentinelAI_Final_Year_CSE_Project_Report.pdf
    API_SPECIFICATION.md
    PRESENTATION_OUTLINE.md
  docker-compose.yml
  README.md
  .gitignore
```

3. Functional and Non-Functional Requirements

ID	Requirement	Expected behavior
FR-01	Event submission	Accept a validated event
FR-02	Event listing	Paginated and filterable records
FR-03	ML analysis	Return anomaly score and category estimate
FR-04	Risk prioritization	Map score to documented risk band
FR-05	Alert review	Open, investigating, resolved statuses
FR-06	Dashboard	Show event counts and risk distribution
FR-07	Audit trail	Record status changes and timestamps

Non-functional requirements: validate inputs; protect secrets; restrict CORS; use parameterized persistence; add authentication before multi-user deployment; paginate queries; handle service failures; minimize personal data; and document retention.

4. Machine-Learning Design

The ML service can combine an unsupervised Isolation Forest for anomaly scoring with a supervised Random Forest classifier for category estimation when reliable labeled data is available.

4.1 Example features

Feature	Meaning	Validation
event_type	Normalized event family	Allowlisted category
source_region	Coarse region label	Avoid precise personal location
failed_login_count	Failures in a time window	Non-negative integer
request_rate	Requests per interval	Finite non-negative number
bytes_sent	Approximate bytes	Non-negative number
hour_bucket	Time bucket	Integer 0–23

4.2 Training and evaluation

Split labeled data into training and held-out test sets. Keep preprocessing in a reproducible pipeline. Report precision, recall, F1-score, confusion matrix, and PR-AUC for imbalanced classification. For anomaly detection, report precision/recall at a selected threshold and explain label provenance. Do not invent accuracy results.

Limitation: synthetic or tiny datasets demonstrate software flow only; they cannot establish production detection quality. Scores are triage signals, not proof of malicious activity.

4.3 Illustrative risk bands

Band	Example score	Suggested handling
Low	0.00–0.39	Trend analysis
Medium	0.40–0.69	Review repeated/correlated events
High	0.70–1.00	Prioritize analyst review

Tune thresholds using validation data. Clearly identify whether a score is an anomaly score, calibrated probability, or heuristic; these are not interchangeable.

5. Database Design

Starter MySQL schema:

```
CREATE DATABASE IF NOT EXISTS sentinelai;
USE sentinelai;

CREATE TABLE security_events (
 id BIGINT PRIMARY KEY AUTO_INCREMENT,
 event_type VARCHAR(80) NOT NULL,
 source_region VARCHAR(80),
 failed_login_count INT NOT NULL DEFAULT 0,
 request_rate DOUBLE NOT NULL DEFAULT 0,
 bytes_sent BIGINT NOT NULL DEFAULT 0,
 hour_bucket TINYINT NOT NULL,
 event_time TIMESTAMP NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE alerts (
 id BIGINT PRIMARY KEY AUTO_INCREMENT,
 event_id BIGINT NOT NULL,
 anomaly_score DOUBLE NOT NULL,
 predicted_category VARCHAR(80),
 risk_level VARCHAR(20) NOT NULL,
 status VARCHAR(24) NOT NULL DEFAULT 'OPEN',
 analyst_note VARCHAR(1000),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
   ON UPDATE CURRENT_TIMESTAMP,
 FOREIGN KEY (event_id) REFERENCES security_events(id)
);
```

```
CREATE TABLE model_runs (
 id BIGINT PRIMARY KEY AUTO_INCREMENT,
 model_name VARCHAR(100) NOT NULL,
 model_version VARCHAR(40) NOT NULL,
 dataset_version VARCHAR(80),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

One event can have multiple alert-analysis records over time. Alerts reference their event. Model-run metadata records model and dataset versions. Add a model_run_id foreign key for stronger traceability.

6. API Specification

Method	Endpoint	Purpose
POST	/api/events	Create event and request analysis
GET	/api/events	List events
GET	/api/events/{id}	Fetch event details
GET	/api/alerts	List alerts and filters
PATCH	/api/alerts/{id}/status	Update analyst status
GET	/api/dashboard/summary	Dashboard counters
GET	/health	Backend health
POST	/predict	ML inference
GET	/ml/health	ML service health

6.1 Example ML request

```
{
  "event_type": "AUTHENTICATION",
  "source_region": "IN-WEST",
  "failed_login_count": 7,
  "request_rate": 18.5,
  "bytes_sent": 12000,
  "hour_bucket": 2
}
```

6.2 Example response

```
{
  "anomaly_score": 0.82,
  "predicted_category": "SUSPICIOUS_AUTH_PATTERN",
  "model_name": "IsolationForest+RandomForest",
  "model_version": "0.1.0"
}
```

The example values are illustrative, not measured results.

7. Frontend Design

Recommended dashboard: overview cards for total events and open/high-risk alerts; event trend chart; risk distribution chart; filterable event table; alert detail panel with features, score, model version, and analyst status; and an analyst review form. Include accessible labels, keyboard navigation, and loading/error states.

Screen	Elements
Dashboard	Summary cards, trends, risk distribution
Events	Search, date range, event type, pagination

Screen	Elements
Alert details	Event context, model score, explanation, status
Model information	Version, training date, evaluation metrics

8. Testing Strategy

Area	Test	Expected result
Backend	Valid event DTO	Validation passes
Validation	Negative request rate	HTTP 400
API integration	Create then retrieve	Record persists
ML unit	Valid feature input	Finite score in documented range
ML validation	Missing/invalid feature	Safe 4xx response
Integration	ML service unavailable	Graceful error
Frontend	Loading/error states	Clear state displayed
End-to-end	Submit event and review alert	Alert and status persist

Run tests and capture actual outputs/screenshots before submission. Do not mark tests as passed until executed.

9. Local Setup Plan

Prerequisites: Java 17+, Maven 3.8+, Python 3.10+, Node.js 18+, MySQL 8+, and Git.

```
# Database
mysql -u root -p < database/schema.sql

# Python ML service
cd ml-service
python -m venv .venv
source .venv/bin/activate
# Windows PowerShell: .venv\\Scripts\\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8001

# Java backend (new terminal)
cd backend
./mvnw spring-boot:run
# Windows: mvnw.cmd spring-boot:run

# React frontend (new terminal)
cd frontend
npm install
npm run dev
```

Suggested local ports: React 5173, Spring Boot 8080, FastAPI 8001. Confirm actual ports in your application configuration.

10. Security, Privacy, and Responsible Use

• Use only synthetic, public, or explicitly authorized event data.

• Never commit passwords, tokens, private logs, or personal data.

• Use environment variables for database credentials and service URLs.

• Add authentication and role-based authorization before external exposure.

• Restrict CORS to the frontend origin and rate-limit exposed APIs.

• Validate request sizes and ranges; return safe error messages.

• Treat model output as advisory and require human review.

• Document data provenance, retention, access, and model limitations.

11. Limitations and Future Enhancements

Limitations may include small labeled datasets, synthetic distributions, false positives, model drift, and missing production-grade authentication or streaming ingestion. Future enhancements include a message queue, explainability, drift monitoring, model registry, role-based access, Docker deployment, CI testing, and privacy-preserving data pipelines.

12. Conclusion

SentinelAI demonstrates a full-stack defensive security-event analytics architecture by integrating React, Java Spring Boot, Python ML, and MySQL. It can form a strong final-year CSE project and placement portfolio when paired with a working implementation, reproducible experiments, honest evaluation, and a clear discussion of limitations.

13. Presentation Outline

Slide	Title	Content
1	Title	Project and student details
2	Problem	Security event volume and analyst workload
3	Objectives	Project goals
4	Proposed system	Manual review vs assisted triage
5	Architecture	React, Spring Boot, FastAPI, MySQL
6	ML approach	Features, Isolation Forest, Random Forest
7	Database/API	Schema and endpoints
8	Demo	Submit sample, view alert, update status
9	Evaluation	Real test evidence and measured metrics
10	Limitations	Responsible use and roadmap
11	Conclusion	Contributions
12	Q&A	Questions

14. GitHub Checklist and Commands

■ Add frontend/, backend/, ml-service/, database/, docs/, README.md, and .gitignore.

■ Include screenshots and synthetic sample records.

■ Document dataset provenance and licensing.

■ Run tests and record actual results.

■ Exclude node_modules, target, .venv, .env, logs, secrets, and private data.

```
git init
git add .
git commit -m "Initial commit: SentinelAI CSE project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/sentinelai.git
git push -u origin main
```

```
# Later updates
git add .
git commit -m "Add dashboard and ML evaluation"
git push
```

Before pushing: Create an empty GitHub repository named sentinelai and replace YOUR-USERNAME with your GitHub username. Never put access tokens in the remote URL.

References

• Spring Boot: https://docs.spring.io/spring-boot/

• Scikit-learn: https://scikit-learn.org/stable/

• FastAPI: https://fastapi.tiangolo.com/

• React: https://react.dev/

• MySQL: https://dev.mysql.com/doc/

• OWASP: https://owasp.org/